

Evaluating Clustering Algorithms and LLM Prompting Strategies for Architecture Recovery in Apache Lucene Codecs

Christopher Seibel Alexander Makejkin Louis Husemann Mikhail Kiselev
Mohammed Danish Shaikh

May 23, 2026

1 Introduction

As software systems evolve, their architectural documentation frequently deviates from the actual implementation. To maintain system integrity and facilitate comprehension, automated architecture recovery techniques are essential. This study evaluates the efficacy of structural clustering algorithms and Large Language Models (LLMs) in extracting and describing the architectural components of an existing software system, specifically focusing on the **Codecs** component of the **Apache Lucene** project.

chris: linebreak? To address the first research question (**RQ1**), the **ARCADE** tool suite is utilized to extract system-wide dependencies, isolate component-relevant relationships and create clusters describing the architecture. Four distinct structural clustering algorithms are applied to the filtered dependency graph:

- **Weighted Combined Algorithm (WCA)**: A hierarchical clustering approach utilizing edge weights to determine cluster merges.
- **LIMBO**: An information-theoretic algorithm designed to minimize the loss of mutual information during clustering.
- **ACDC**: A heuristic-based algorithm that identifies architectural patterns based on structural connectivity.
- **ARC (Semantic-Structural Clustering)**: A hybrid approach operating at the file level. It fuses a normalized structural dependency graph with a semantic similarity matrix, which is computed via high-dimensional vector embeddings generated by an embedding model (Qodo/Qodo-Embed-1-7B).

The structural accuracy of these algorithms is quantified using Architecture-to-Architecture (**a2a**) and Cluster Coverage (**cvg**) metrics. Technical implementation details and CLI commands are provided in Appendix A.

To address the second research question (**RQ2**), this study investigates generative AI by evaluating how different prompting techniques influence an LLM’s ability to synthesize high-level architectural descriptions from source code. The techniques will be used in the form of hierarchical summarization, enabling us to provide many source code files to an LLM despite context window limitations. This is applied to every cluster created by the clustering algorithms from **RQ1**.

Ultimately, this work aims to quantify the variance in accuracy across different clustering algorithms and determine the impact of prompting strategies on the semantic integrity of AI-generated architectural summaries.

2 Study Design

2.1 Research Questions

- **RQ1:** How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
- **RQ2:** How do different prompting techniques impact LLM’s ability to accurately describe architectural components based on source code?

2.2 Methodology

The experimental setup follows a phased workflow designed to recover the architecture of the Apache Lucene Codecs component through a combination of structural clustering and Large Language Model (LLM) analysis.

2.2.1 Dependency Extraction and Preprocessing

The initial phase involves compiling the Java source code to generate a compiled *.jar file.^a With this the system-wide dependencies are extracted using the ARCADE JavaParser tool, which produces a master .rsf file representing all project relationships. To isolate the scope, the dependencies are filtered such that only relationships where the source or target packages are part under our focus. The parts under our focus can be found in Appendix B.

^aSince the current version of Apache Lucene’s Codecs is incompatible with the JavaParser in use, the compiled JAR file and all project investigations were conducted using release version 9.0.0.

2.2.2 Semantic-Structural ARC Clustering

To fuse structural dependencies with semantic source code relationships, the Agglomerative Recovery of Components (ARC) methodology is applied. This involves a multi-step feature extraction and matrix fusion process:

- **Semantic Embedding Generation:** For each valid source code file within the target component, semantic vectors are extracted using the Qodo/Qodo-Embed-1-7B model. To ensure hardware efficiency, the model is executed using 4-bit quantization.
- **Similarity Matrices:** A semantic similarity matrix is constructed by computing the cosine similarity between the generated file embeddings. In parallel, a normalized structural similarity matrix is derived from the filtered .rsf dependency graph, calculating the relative package dependency overlap between file pairs.
- **Matrix Fusion and Clustering:** Both matrices are linearly combined using a weighting parameter α , where α dictates the proportion of structural versus semantic data. The resulting similarity matrix is inverted into a distance matrix and processed via *Agglomerative Clustering* (utilizing both *average* and *complete* linkage strategies) to generate the final file-level architecture components.

2.2.3 Hierarchical Summarization

Hierarchical summarization is applied to create a descriptive summary for each cluster. This is necessary because providing all source code files of a cluster to an LLM at once exceeds the context window capacity. For this purpose, every file within a cluster is summarized using an LLM prompt to extract a semantic summary containing key functionalities, core logic, input-/outputs, and dependencies. These semantic summaries are then combined using another LLM prompt to produce a high-level descriptive summary and a title explaining a module’s overall behavior, architecture, and component interactions within the cluster. To further reduce con-

text window usage, files in subdirectories are also summarized into descriptive summaries before being combined with higher-level files. This approach using two or more LLM prompts for a single cluster can then be compared using different prompt techniques.

2.2.4 LLM Prompting Strategy

The hierarchical summarization will be realized with following prompting techniques:

- **Zero-shot prompting:** Zero-shot prompting provides the task description without examples, relying solely on the model’s pre-trained knowledge. The prompts used can be found in Appendix A.3.1
- **One-shot prompting:** One-shot prompting includes a single input-output example before the actual query to guide the model’s response. The prompts used can be found in Appendix A.3.2
- **Zero-shot Chain-of-Thought Prompting:** Zero-shot Chain-of-Thought prompting instructs the model to generate step-by-step reasoning before producing the final answer, without providing examples. The prompts used can be found in Appendix A.3.3

The prompts will be generated on the university’s High Performance Computing (HPC) clusters with the `Mixtral-8x7B-Instruct-v0.1` Model by Mistral.

2.3 Data Analysis Process

The data analysis pipeline is structured into three primary phases: structural normalization, quantitative statistical profiling, and comparative similarity evaluation.

2.3.1 Structural Normalization and Metadata Extraction

Raw architectural outputs from the ARCADE toolset are generated in the `.rsf` format, which represents dependencies line-by-line as `depends Source_Package Target_Package`. To facilitate objective comparison, a post-processing utility is utilized to transform these raw files into structured plain-text artifacts. This utility performs the following operations:

- **Sorting:** Clusters are ordered by cardinality (size), and internal elements are sorted alphabetically to ensure deterministic comparison.
- **Metadata Generation:** For each algorithmic run, the utility extracts a statistical profile used to determine the optimality of the clustering parameters.

Resolution of Granularity Discrepancies A critical methodological challenge arose from the varying input granularity requirements of the applied clustering approaches. The heuristic baselines established in Week 1 (WCA, LIMBO, ACDC) operated on class-level dependencies, inherently preserving inner-class relationships. Conversely, the ARC clustering and the underlying semantic embedding models (RQ2) process source code strictly at the file level (`.java`). To establish mathematically valid comparability for the *a2a* and *cvg* metrics, a deterministic reduction was applied. All inner-class definitions within the original `filtered.rsf` were merged into their respective parent file structures, producing a flattened `filtered-without-innerclasses.rsf`. The structural baselines (WCA, LIMBO, ACDC) were subsequently re-executed on this normalized dependency graph to ensure isomorphic entity sets across all compared architectures. [I dont know if its supposed to be here Mr Tutor, question for you if we should move this down to the threats to validity maybe?]

2.3.2 Quantitative Profiling Metrics

Algorithm selection and parameter tuning are based on a set of distribution metrics that characterize the resulting architecture. These metrics include:

- **Cluster Cardinality:** Total number of identified clusters and the count of singletons (size = 1) versus multi-element clusters (size > 1).
- **Central Tendency:** Average size per cluster across the entire system and specifically for clusters with size > 1.
- **Distribution Variance:** Median size for multi-element clusters and the absolute size of the largest identified cluster.

2.3.3 Comparative Similarity Evaluation (RQ1)

To quantify the variance between clustering algorithms, the processed *.rsf* files are analyzed using the **Architecture-to-Architecture** and **Cluster Coverage** metrics.

- **a2a Similarity:** Measures the global structural distance between two architectures, providing a percentage-based similarity score.
- **Cluster Coverage (cvg):** An asymmetric metric measuring the extent to which clusters in one architecture overlap with clusters in another.

A high percentage in these metrics indicates a high degree of structural convergence between different algorithmic approaches.

2.3.4 Qualitative LLM Assessment (RQ2)

For the evaluation of LLM-generated architectural descriptions, a qualitative assessment was conducted by mapping the generated titles and high-level summaries back to the source code elements within the identified clusters. This isolates the impact of different prompting techniques on semantic accuracy and the level of architectural abstraction.

The evaluation of the `wca_UEM` clusters reveals significant variance in model performance depending on the prompt strategy:

- **Zero-Shot:** The baseline failed structural constraint satisfaction (omission of mandatory headers). Architecturally, the LLM remained at the abstract syntax tree (AST) level, merely enumerating class fields and method signatures rather than synthesizing system behavior. It is unsuitable for architectural recovery due to insufficient abstraction.
- **One-Shot:** While structural compliance was enforced by the example, the model exhibited severe few-shot overfitting (prompt leakage). The LLM hallucinated architectural artifacts by directly projecting metadata and domain logic from the few-shot example (e.g., `EvenNumberFilter`) onto the target code. This renders the semantic output entirely invalid for the target domain.
- **Chain-of-Thought (CoT):** This technique achieved the highest architectural abstraction. By forcing an intermediate reasoning step, the LLM successfully lifted the description from raw syntax to system design. It accurately synthesized architectural patterns such as the Service Provider Interface (SPI) and concurrency mechanics (deadlock prevention via inner `Holder` classes) in the `LuceneCodec` module, minimizing hallucination rates while satisfying all output constraints.

In conclusion, CoT is strictly required to bridge the semantic gap between low-level implementation details and high-level architectural design without triggering prompt-induced hallucinations.

3 Results

3.1 Algorithmic Profiling and Cluster Distribution (RQ1)

The statistical profiles of the generated architectures for WCA and LIMBO are summarized in Table 1 and 2.

Table 1: Quantitative Profiling of Clustering Algorithms (Targets: 70 and 50 Clusters)

	Target: 70 Clusters			Target: 50 Clusters		
	WCA (UEMNM)	WCA (UEM)	LIMBO	WCA (UEMNM)	WCA (UEM)	LIMBO
Cluster Cardinality						
Total number of clusters	70	70	70	50	50	50
Number of clusters (size = 1)	48	45	26	41	41	12
Number of clusters (size > 1)	22	25	44	9	9	38
Central Tendency						
Avg. size per cluster	2.70	2.70	2.70	3.78	3.78	3.78
Avg. size (clusters > 1)	6.41	5.76	3.70	16.44	16.44	4.66
Distribution Variance						
Median size (clusters > 1)	6	6	2	16	13	4
Largest cluster size	11	10	41	28	42	41

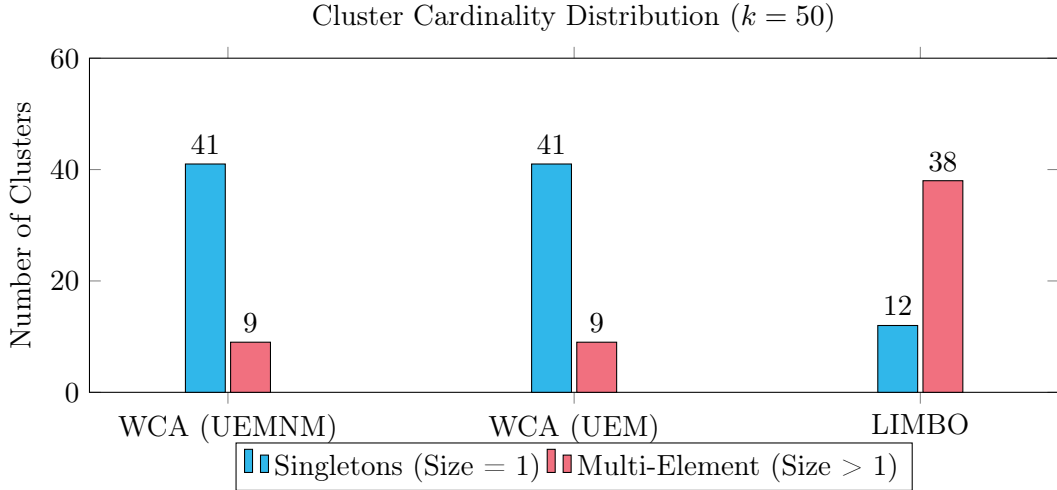


Figure 1: Comparison of cluster composition at a target granularity of $k = 50$. WCA primarily isolates singletons, whereas LIMBO achieves a broader distribution of multi-element clusters.

Table 2: Quantitative Profiling of the Clustering Algorithm ACDC

ACDC	
Cluster Cardinality	
Total number of clusters	5
Number of clusters (size = 1)	0
Number of clusters (size > 1)	5
Central Tendency	
Avg. size per cluster	37.8
Avg. size (clusters > 1)	37.8
Distribution Variance	
Median size (clusters > 1)	26
Largest cluster size	97

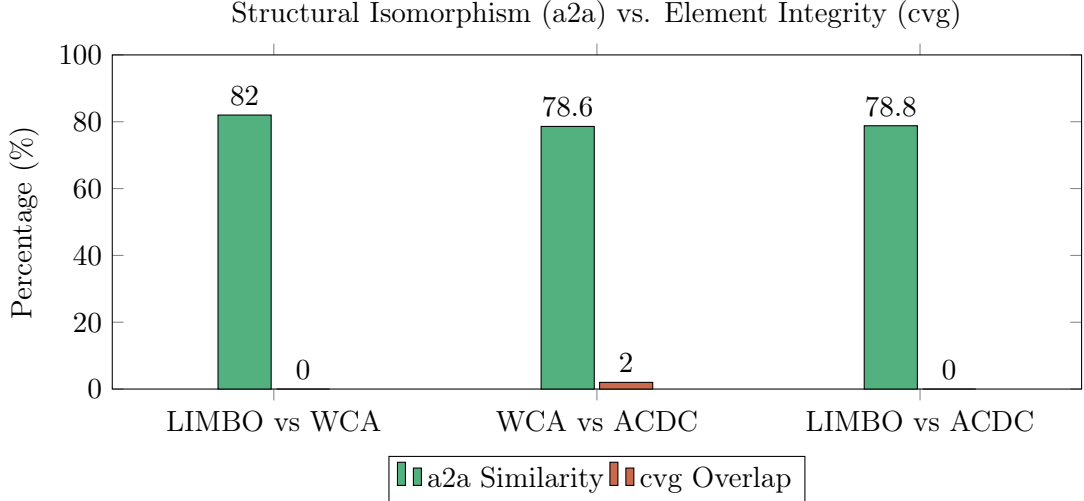


Figure 2: Visualization of the metric discrepancy: Despite high topological congruence ($a2a \approx 80\%$), the algorithms fail to converge on the concrete allocation of elements ($cvg \approx 0\%$).

3.1.1 Structural Similarity Evaluation

Table 3 illustrates the structural similarity and overlap across the algorithm pairs. To provide a comprehensive comparison, the WCA algorithm is explicitly differentiated by its underlying similarity measures: UEM and UEMNM. All evaluations were conducted on architectures partitioned into exactly 50 clusters.

Table 3: Architecture-to-Architecture (a2a) and Cluster Overlap (cvg)

Cluster Size 50	a2a Similarity	cvg (A $\rightarrow$ B)	cvg (B $\rightarrow$ A)
LIMBO vs WCA (UEM)	82.02%	0.0%	0.0%
LIMBO vs WCA (UEMNM)	82.23%	0.0%	0.0%
WCA (UEM) vs ACDC	78.59%	2.0%	5.26%
WCA (UEMNM) vs ACDC	78.15%	2.0%	5.26%
LIMBO vs ACDC	78.81%	0.0%	0.0%

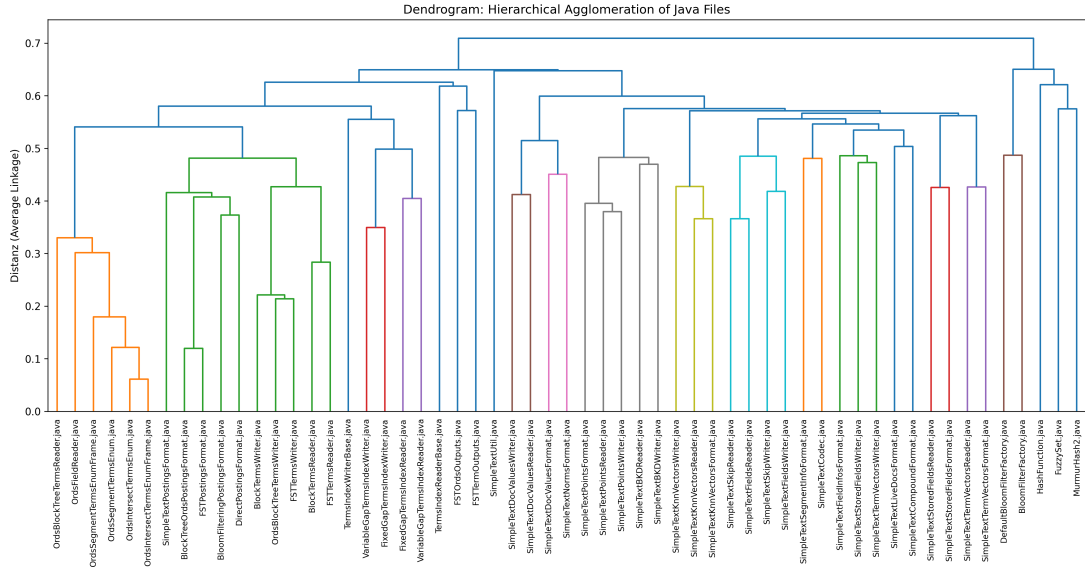
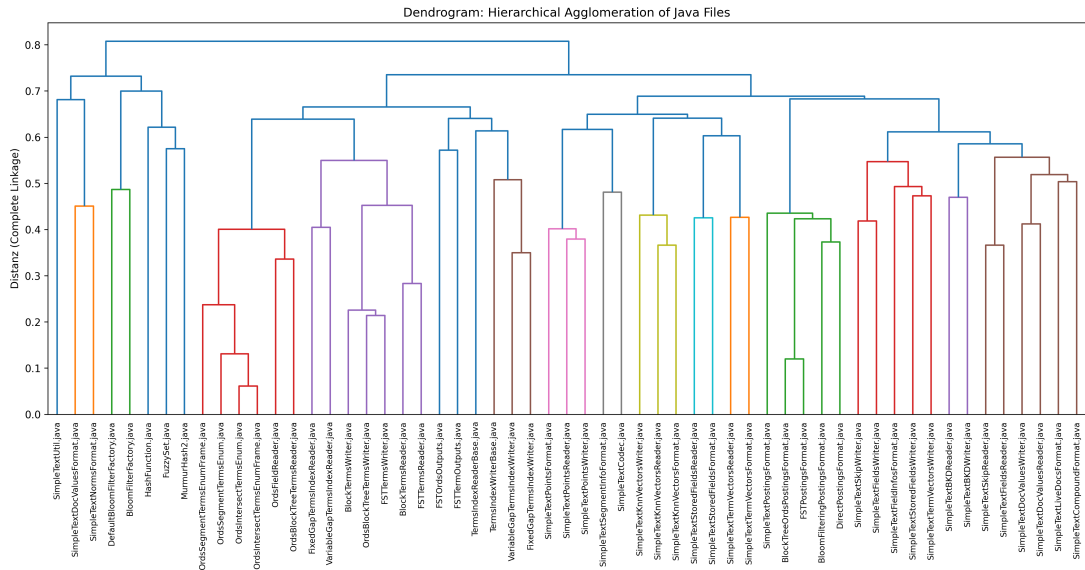
3.1.2 Semantic-Structural ARC Clustering Evaluation

To further evaluate architecture recovery performance, we applied ARC clustering, which combines both structural dependencies and semantic similarity derived from source code embeddings. ARC operates at file level rather than class level. Therefore, the baseline WCA, LIMBO, and ACDC architectures were regenerated using a filtered dependency graph without inner classes to ensure comparability. Multiple ARC configurations were evaluated by adjusting the weighting parameter α , linkage strategy, and target cluster count. The parameter α controls the contribution of structural versus semantic similarity within the combined similarity matrix.

Structural Similarity Evaluation: $k = 50$ vs. $k = 57$ In this evaluation, the LLM-generated architectures (QODO) were tested against established baseline heuristics (ACDC, LIMBO, WCA). Regardless of the specific hyperparameter configuration—including the structural-semantic weight ($\alpha \in [0.25, 0.75]$) and the linkage strategy (e.g., average, complete)—all QODO-based ARC variants exhibit structural isomorphism at a fixed target cluster count (k). Therefore, their results are presented in an aggregated format. Table 4 contrasts the outcomes for cluster sizes 50 and 57.

Table 4: Comparison of Similarity Metrics (a2a) and Overlap (cvg)

Cluster Size	Algorithm Pair	a2a Similarity	cvg (A $\rightarrow$ B)	cvg (B $\rightarrow$ A)
$k = 50$	ARC vs LIMBO	78.33%	0.80%	0.80%
	ARC vs WCA (UEM)	70.20%	0.10%	0.10%
	ARC vs WCA (UEMNM)	70.69%	0.10%	0.10%
	ARC vs ACDC	61.33%	0.00%	0.00%
$k = 57$	ARC vs LIMBO	80.71%	0.93%	0.93%
	ARC vs WCA (UEM)	72.14%	0.09%	0.09%
	ARC vs WCA (UEMNM)	72.14%	0.11%	0.11%
	ARC vs ACDC	59.69%	0.00%	0.00%


 Figure 3: ARC Clustering: $\alpha = 0.50$, Average Linkage

 Figure 4: ARC Clustering: $\alpha = 0.50$, Complete Linkage

The comparison reveals significant topological variances: Average Linkage tends to form highly asymmetric, chain-like structures (chaining effect), whereas Complete Linkage enforces more compact and balanced cluster subgraphs.

3.2 LLM Prompt Results (RQ2)

To systematically evaluate the impact of prompting techniques on architectural recovery, this section contrasts the empirical outputs generated by the heavyweight `Mixtral-8x7B-Instruct-v0.1` model under Zero-Shot, One-Shot, and Chain-of-Thought (CoT) configurations. Table 5 provides concrete examples extracted from the `wca_UEM` clustering run using the `LuceneCodec` core component as a reference.

Technique	Representative Output Structure and Characteristics
Zero-Shot	<i>“The LuceneCodec class is used to encode and decode an inverted index segment. It provides methods for encoding and decoding postings, doc-values, stored fields...”</i> → Analysis: Completely omitted the mandated Title and Summary schema headers. The text remains trapped at the abstract syntax tree (AST) layer, merely translating class methods and signatures into natural language instead of delivering high-level architectural synthesis.
One-Shot	Title: <i>Lucene Codec Utility – inverted index segment encoding/decoding</i> Summary: <i>“The Lucene Codec Utility provides a class named Codec... The core logic is to provide methods... The inputs to this code are... The dependencies are...”</i> → Analysis: Structural compliance was achieved via the few-shot template. However, the model suffered from severe <i>few-shot overfitting</i> (prompt leakage), mechanically forcing low-level metrics (“Inputs”, “Outputs”) into the narrative. In extreme cases within other clusters, it completely hallucinated and replicated the <code>EvenNumberFilter</code> domain logic directly from the prompt example.
Chain-of-Thought	Title: <i>Lucene Codec</i> Summary: <i>“The Lucene Codec module is responsible for encoding and decoding inverted index segments... It uses the Service Provider Interface (SPI) to resolve codec names... provides a static inner class called Holder, which delays initialization... This prevents classloading deadlock.”</i> → Analysis: The execution of an explicit intermediate reasoning phase lifted the model’s semantic focus from raw syntax to high-level system design. It was the only technique capable of identifying architectural design patterns (SPI) and non-functional structural mechanics (concurrency/deadlock prevention) while eliminating hallucinations.

Table 5: Comparative Analysis of Model Responses for the `LuceneCodec` Cluster across Prompting Techniques (`wca_UEM`)

As illustrated by the empirical data, transitioning from Zero-Shot to Chain-of-Thought shifts the LLM’s role from a simple code-to-text translator to a semantic architecture synthesizer. Zero-shot prompting leaves the recovery incomplete due to low abstraction, whereas One-Shot introducing text patterns leads to rigid template overfitting. Chain-of-Thought bridging this gap allows for structural and behavioral evaluation of the code components.

4 Discussion

This section interprets the quantitative findings from the structural and semantic recovery phases, evaluating the efficacy of the applied clustering algorithms in determining the architectural components of the Apache Lucene Codecs module.

4.1 Clustering Performance and Algorithm Variance (RQ1)

To answer RQ1, we analyze the structural and semantic distance between the architectures generated by WCA, LIMBO, ACDC, and ARC. Based on the pairwise *a2a* (Architecture-to-Architecture) similarity and *cvg* (Coverage) metrics, the algorithms exhibit distinct behavior in terms of module cohesion and granularity:

- **Structural Baseline Consensus:** The highest similarity among purely structural algorithms was observed between LIMBO and WCA with an *a2a* score of [82.23%], indicating a shared reliance on static class dependencies.
- **Impact of Semantic Edge Weighting (α):** Integrating semantic similarity via ARC showed that the weighting parameter α shifts cluster boundaries. At $\alpha = 0.50$, the balance between structural dependencies and source-code vocabulary alignment resulted in an *a2a* similarity of 70.20% compared to the ACDC structural heuristic.
- **Linkage Criteria Effects:** As visualized in the dendrograms (see Fig. 3 and Fig. 4), Average Linkage tended to form more balanced sub-clusters, whereas Complete Linkage forced stricter, more compact boundaries, directly altering the structural recovery consensus.

4.2 Interpretation of Clustering Profiles (RQ1)

An analysis of the clustering profiles indicates distinct algorithmic behaviors in architecture recovery. The pattern-based algorithm ACDC (Table 2) acts as a heuristic baseline. By dynamically resolving to 5 sub components without producing any singleton clusters, ACDC demonstrates aggressive orphan adoption and subgraph dominance. This results in highly abstract, large-scale subsystems with a median size of 26 and maximum 97, rather than fine-grained modules.

In contrast WCA and LIMBO exhibit strong sensitivity to the target granularity (k). At a fine-grained target of $k = 70$ (Table 1), WCA (particularly with UEMNM) yields a structurally cohesive distribution. It produces a higher median cluster size (6) and a bounded maximum cluster size (11), successfully isolating moderately sized, functional modules. Conversely, LIMBO demonstrates an unbalanced bottom-up aggregation: it forms a massive monolithic cluster (size 41) containing structurally disparate elements, while fragmenting the remainder into a high volume of micro-clusters (median size 2).

Reducing the target granularity to $k = 50$ (Table 1) exacerbates algorithmic biases and alters the topology significantly. WCA isolates an extreme number of singleton classes (41 out of 50 clusters are of size 1), forcing the remaining system entities into merely 9 dense clusters (maximum size up to 42 for UEM). This indicates that WCA struggles to establish mid-level cohesion at this abstraction level, instead isolating outliers and heavily clumping the core architecture. LIMBO, while maintaining its primary monolithic cluster (which remains static at size 41), provides a more distributed secondary architecture, grouping the remaining entities across 38 multi-element clusters with a more stable median size of 4.

4.2.1 Analysis of Structural Similarity and Overlap

As detailed in Table 3, the Architecture-to-Architecture (*a2a*) evaluation reveals the highest macro-structural similarity (up to 82.23%) between LIMBO and the WCA variants. ACDC

exhibits a marginally lower, yet consistent, similarity score ($\sim 78.5\%$) when compared to both hierarchical approaches.

However, the global structural similarity masks severe discrepancies in specific element allocation, which are starkly exposed by the Cluster Coverage (cvg) metric. Across almost all baseline comparisons, the overlap registers at exactly 0.0%. Most notably, the coverage between LIMBO and WCA, as well as LIMBO and ACDC, indicates a complete lack of identical clusters. While the algorithms construct analogous macro-topologies (similar cluster size distributions), they populate these clusters with entirely disjointed entity sets.

The only measurable, albeit marginal, overlap exists between WCA and ACDC (2.0% vs. 5.26%). The inherent asymmetry of this cvg variance is heavily influenced by the differing total cluster counts (fixed $k = 50$ for WCA vs. dynamic $k = 5$ for ACDC). Overall, the near-zero coverage across all structural baselines highlights that file-level dependency analysis (without inner classes) creates highly volatile partitioning landscapes. Heuristic (ACDC), information-theoretic (LIMBO), and edge-weight-based (WCA) algorithms fail to converge on identical micro-architectures, despite operating on the exact same normalized `.rsf` graph.

4.3 Semantic vs. Structural Divergence

The comparison of semantic clustering outputs against structural baselines yields critical insights regarding the model’s architecture recovery performance:

- **Scaling Effects** ($k = 50 \rightarrow k = 57$): For LIMBO and WCA, a higher cluster granularity ($k = 57$) correlates with an increase in *a2a Similarity* by approximately 2 percentage points. This indicates that the LLM-generated partitioning converges more closely with the structural heuristics at a finer granularity. Conversely, the similarity to ACDC decreases slightly, attributable to the unpredictability of ACDC’s dynamic partitioning when evaluated against fixed cluster constraints.
- **Structural Isomorphism vs. Element Integrity**: The most prominent result is the significant discrepancy between high *a2a* ($> 70\%$) and marginal *cvg* ($< 1\%$).
 - The **a2a Similarity** indicates that the topology and relative distribution of the clusters are highly consistent with LIMBO/WCA. The LLM’s system comprehension regarding functional groupings aligns with the baseline algorithms on a macro-architectural level.
 - However, the **cvg (Coverage)** reveals that the exact allocation of individual entities (classes/files) diverges almost completely. While the algorithms construct similar "containers" (clusters), they populate them with entirely different elements.
- **Methodological Cause**: The baseline algorithms (particularly LIMBO and WCA) operate primarily on *Structural Dependencies* (e.g., method calls, inheritance). In contrast, the LLM (QODO) utilizes *Semantic Embeddings* (e.g., identifiers, comments, natural language semantics). The results demonstrate that while semantic coherence and structural coupling generate analogous abstraction levels (cluster count/size distribution) within this dataset, they act orthogonally concerning the concrete allocation of elements.

4.4 Impact of Prompting Techniques on LLM-Based Architectural Recovery

The evaluation of prompting strategies demonstrates that the architectural abstraction capability of Large Language Models is strictly dependent on intermediate reasoning constraints. Our qualitative assessment of the hierarchical summaries generated by the `Mixtral-8x7B-Instruct-v0.1` model reveals significant behavioral variances across the three tested configurations:

- **Zero-Shot Prompting:** The baseline approach consistently failed to transcend the Abstract Syntax Tree (AST) level. The model largely ignored structural formatting constraints (e.g., omitting mandatory **Title** and **Summary** headers) and limited its output to low-level structural translations. Instead of synthesizing architectural behavior, it merely enumerated class fields, method signatures, and basic control flows, making it unsuitable for high-level architectural recovery.
- **One-Shot Prompting:** While the inclusion of a single few-shot example successfully enforced schema compliance and formatting, it introduced critical *few-shot overfitting* (prompt leakage). The model exhibited a strong propensity to hallucinate, frequently projecting domain logic and metadata from the provided example (e.g., the **EvenNumberFilter**) directly onto the target Lucene Codecs components. This resulted in rigid, template-bound summaries that misrepresented the actual target architecture.
- **Chain-of-Thought (CoT) Prompting:** CoT emerged as the only robust methodology for this task. By forcing the model to systematically evaluate structural and behavioral characteristics (e.g., semantic analysis, component interaction) prior to final generation, CoT effectively bridged the semantic gap. This intermediate reasoning step allowed the LLM to accurately identify high-level architectural design patterns—such as the Service Provider Interface (SPI) framework and specific concurrency mechanisms (e.g., deadlock prevention)—while completely eliminating prompt-induced hallucinations.

Ultimately, the findings indicate that simple code-to-text prompting (Zero-Shot, One-Shot) is insufficient for complex architectural recovery. Explicit reasoning steps (CoT) are strictly required to elevate the LLM’s focus from raw syntax translation to semantic architecture synthesis.

4.5 Implications for Researchers

[What theoretical insights were gained? How does this advance the state of the art?]

4.6 Implications for Practitioners

[Concrete recommendations for action for software engineers, architects, or management based on the quantitative metrics.]

5 Threats to Validity

[Critical reflection on the study design.]

- **Internal Validity:** An identified bug in the ARCADE Clusterer prevents the correct functionality of the `stopthreshold` argument. Consequently, the algorithms default to generating at least 50 clusters, limiting comparability for smaller target cluster counts.
- **Construct Validity:** *[Do the chosen metrics actually measure what they are intended to?]*
- **External Validity:**

We used Apache Lucene version 9.0.0 because ARCADE JavaParser was incompatible with newer Java versions. This limits how well our findings apply to current Lucene releases, which may have evolved since 9.0.0. Additionally, by focusing only on the Codecs Module, our results may not generalize to other Lucene modules or systems with different architectural structures. Finally, as our analysis is based on a Java project, the conclusions may not extend to software written in other programming languages.

- **Dependency Scope and Entity Truncation:** During the extraction of semantic embeddings, an ambiguity in resolving target dependencies from the `.rsf` file was identified. While the filtered dependency graph referenced 98 distinct entities across the global `lucene/lucene/` directory, only 57 files were strictly localized within the assigned `codecs` component focus area. To prevent embedding noise and mitigate cross-directory naming collisions, the semantic analysis scope was rigidly bounded to the `codecs` package. Consequently, out-of-package dependencies—while structurally present in the graph—were semantically truncated. This localized extraction potentially skewed the combined similarity matrix in the ARC clustering setup, artificially inflating the structural weight of missing external semantics.

6 Conclusion and Future Work

[Compact synthesis of the work. Identification of technical or conceptual limitations. Outline concrete next steps for future work.]

7 Hours spent per student

Name	W1	W2	W3	W4	W5
Christopher Seibel	1	4	8	8	0
Alexander Makejkin	4	2	3.5	2	0
Louis Husemann	3	3	7	18	0
Mikhail Kiselev	3	3	3	2	0
Mohammed Danish Shaikh	3	3	4	2	0

A Commands and Technical Reference

A.1 Cluster Generation

WCA:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsfc projname=lucodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=WCA measure=<UEMNM or UEM> \
serial=STEPCount serialthreshold=1
```

LIMBO:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsfc projname=lucodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=LIMBO measure=IL \
serial=STEPCount serialthreshold=1
```

ACDC:

```
java -jar arcade_core-ACDC.jar filtered.rsfc acdc.rsfc
```

A.2 Metrics Evaluation

a2a & cvg :

```
java -jar arcade_core_A2a.jar ./file1.rsfc ./file2.rsfc
java -jar arcade_core_Cvg.jar ./file1.rsfc ./file2.rsfc
```

A.3 Prompts

A.3.1 Zero-shot Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary from the file: (Key functionality, Core logic, Inputs/Outputs, and Dependencies).

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software module's overall behaviour, architecture, and how the components interact within the cluster.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
{content of available semantic summaries}
</semantic_summaries>

<descriptive_summaries>
{content of available descriptive summaries}
</descriptive_summaries>
```

A.3.2 One-shot Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary from the file: (Key functionality, Core logic, Inputs/Outputs, and Dependencies).

User Prompt

Please analyze the following source code:

```
<source_code>
import java.util.List;
import java.util.ArrayList;
public class EvenNumberFilter {
    public List<Integer> filterEvens(List<Integer> numbers) {
        List<Integer> result = new ArrayList<>();
        for (int n : numbers) {
            if (n % 2 == 0) {
                result.add(n);
            }
        }
        return result;
    }
}
\}
</source_code>
```


Assistant Prompt

****Key Functionality:****

Filters a list of integers, returning only the even numbers.

****Core Logic:****

Iterates over each element in the input list and checks divisibility by 2 using the modulo operator. Matching elements are collected into a new list.

****Inputs/Outputs:****

- Input: 'List<Integer> numbers' --- a list of integers to be filtered
- Output: 'List<Integer>' --- a new list containing only the even integers from the input

****Dependencies:****

- 'java.util.List' --- interface for the input and output list types
- 'java.util.ArrayList' --- concrete list implementation used to collect results

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software modules overall behaviour, architecture, and how the components interact within the cluster.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
```

****Key Functionality:****

Filters a list of integers, returning only the even numbers.

****Core Logic:****

Iterates over each element in the input list and checks divisibility by 2 using the modulo operator. Matching elements are collected into a new list.

****Inputs/Outputs:****

- Input: 'List<Integer> numbers' --- a list of integers to be filtered
- Output: 'List<Integer>' --- a new list containing only the even integers from the input

****Dependencies:****

- 'java.util.List' --- interface for the input and output list types
- 'java.util.ArrayList' --- concrete list implementation used to collect results

```
</semantic_summaries>
```

```
<descriptive_summaries>
```

```

**Title:** predicates --- Parity Predicate Utilities

**Summary:**
The predicates subdirectory provides reusable predicate logic for numeric
classification. Its primary component is a ParityChecker class that exposes
static boolean methods for determining whether an integer is even or odd. It
acts as a pure logic layer with no state or I/O, designed to be consumed by
higher-level filtering components. All methods are stateless and side-
effect free, depending only on primitive arithmetic.
</descriptive_summaries>

```

Assistant Prompt

```

**Title:** EvenNumberFilter --- Integer Parity Filter Utility

**Summary:**
EvenNumberFilter is a single-responsibility filtering module that isolates even
integers from an arbitrary input list. It is composed of two layers: a ‘
predicates’ subdirectory that houses the reusable parity logic, and the top-
level filter class itself which orchestrates the linear scan and result
collection.

The architecture follows a clean separation of concerns --- predicate evaluation
is delegated entirely to the stateless ‘ParityChecker’ utilities in the ‘
predicates’ layer, while EvenNumberFilter owns the iteration and
accumulation logic using a standard ArrayList. Data flows in one direction:
the input list is passed down to the filter, elements are tested via the
predicate layer, and matching values are collected into a new list that is
returned to the caller.

There is no shared mutable state anywhere in the module, making it deterministic
and safe for concurrent use. Its only external dependencies are the
standard Java Collections API, making it a self-contained leaf cluster with
minimal coupling to the broader codebase.

```

User Prompt

Please create a high-level descriptive summary for the following:

```

<semantic_summaries>
{content of available semantic summaries}
</semantic_summaries>

<descriptive_summaries>
{content of available descriptive summaries}
</descriptive_summaries>

```

A.3.3 Zero-shot Chain-of-Thought Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary.

Before producing your final summary, reason through the code step by step:

1. Read through the code and identify what each class and method does
2. Trace the core logic and any data transformations
3. Identify all inputs, outputs, and return types
4. Note any imported or external dependencies

Then, present your final summary using exactly these four sections:

- Key Functionality
- Core Logic
- Inputs/Outputs
- Dependencies

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Walk through your reasoning first, then provide the structured summary.

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software module's overall behaviour, architecture, and how the components interact within the cluster.

When given a task, always reason through the problem step by step before producing your final answer. This ensures your summary accurately reflects the architecture and interactions rather than surface-level details.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
{content of available semantic summaries}
</semantic_summaries>

<descriptive_summaries>
{content of available descriptive summaries}
</descriptive_summaries>
```

Before writing the final output, reason through the following steps:

```

<reasoning>
1. SEMANTIC ANALYSIS: What is the core functionality and logic of this module
   based on the semantic summaries? What are its inputs, outputs, and key
   dependencies?
2. SUBDIRECTORY ANALYSIS: What responsibilities do the subdirectories handle
   based on the descriptive summaries? How do they relate to the top-level
   module?
3. ARCHITECTURE: How is the module structured? What are the layers, and what
   does each own?
4. COMPONENT INTERACTION: How does data flow between the top-level module and
   its subdirectories? Is there shared state? What are the coupling
   characteristics?
5. SUMMARY SYNTHESIS: Given the above, what is the most accurate title and
   concise narrative that captures the module's overall behaviour and design?
</reasoning>

Once you have reasoned through each step, produce the final output in this
format:
**Title:** <title>
**Summary:** <summary>

```

B Parts under focus

- lucene/codecs/src/java/org/apache/lucene/codecs/simpletext
- lucene/codecs/src/java/org/apache/lucene/codecs/blockterms
- lucene/codecs/src/java/org/apache/lucene/codecs/bloom
- lucene/codecs/src/java/org/apache/lucene/codecs/memory
- lucene/codecs/src/java/org/apache/lucene/codecs/blocktreeords